

Teoria della dimostrazione, Teoria dei tipi, Proof Assistants e AI

verso una nuova pratica della matematica

Marino Miculan

Scuola Estiva di Logica AILA 2026

Bardonecchia, 31 agosto – 5 settembre 2026

DMIF, Università di Udine | marino.miculan@uniud.it | <https://marino.miculan.org>

LEZIONE 3

Martin-Löf Type Theory e il Calcolo delle Costruzioni Induttive

AILA 2026 — Bardonecchia

Obiettivo

Arrivare al nucleo teorico **esatto** su cui sono costruiti Lean 4 e Rocq, così che il laboratorio delle lezioni successive non sia una scatola nera.

In questa lezione

1. Lezione 3 — MLTT e CIC
 - 1.1 Da dove viene MLTT
 - 1.2 L'apparato giudizionale
 - 1.3 Tipi Π e Σ
 - 1.4 Uguaglianza proposizionale
 - 1.5 Universi
 - 1.6 Tipi induttivi ed eliminatori
 - 1.7 Che cosa si sa di MLTT
 - 1.8 Che aspetto ha un modello

Blocco 1

Martin-Löf Type Theory

Dopo il cubo: cambiare programma

Nel cubo si parte da $\lambda \rightarrow$ e si aggiungono assi. MLTT non è un altro vertice: è un **programma diverso**.

- ▶ l'obiettivo di Martin-Löf non è un calcolo più espressivo, ma una **fondazione costruttiva della matematica**, alternativa alla teoria degli insiemi;
- ▶ nella pratica usuale ci sono *due* livelli — una logica del prim'ordine e, sopra, un universo di insiemi. Qui il livello è **uno solo**;
- ▶ una proposizione **è** un tipo: non per un isomorfismo  L1: *Curry–Howard*, ma per definizione.

💡 Un linguaggio, tre mestieri

Lo stesso formalismo è insieme *logica*, *linguaggio di programmazione* e *fondazione*. E le regole non si postulano: si giustificano spiegando *che cosa significa* sapere che a è un elemento di A — la BHK di L1, promossa a semantica ufficiale.

- ▶ **1971** — prima versione, con $\text{Type} : \text{Type}$. **Incoerente**: è il bersaglio del paradosso di Girard  *L2: ce ne siamo appena occupati*;
- ▶ **1972–73** — versione **predicativa**: entra la gerarchia di universi. È la MLTT di cui parleremo;
- ▶ **1980** — lezioni a Padova, raccolte da Giovanni Sambin; diventano *Intuitionistic Type Theory*, Bibliopolis 1984.

Due varianti, ancora oggi

Estensionale: l'uguaglianza proposizionale si riflette in quella giudizionale — comodo, ma il type checking diventa indecidibile (NuPRL). **Intensionale**: le due restano distinte, il controllo resta decidibile. È la strada di Agda, Idris, Rocq e Lean.

Che cosa cambia rispetto a λC

	λC , il vertice del cubo	MLTT
si parte da	un λ -calcolo da estendere	i giudizi
le proposizioni	sono tipi, per Curry–Howard	<i>sono</i> tipi, per definizione
l'uguaglianza	un giudizio	anche un tipo
gli universi	Prop impredicativo	gerarchia predicativa
i dati	si codificano  L2: $\forall X. \dots$	tipi induttivi primitivi

Le ultime tre righe sono il programma del blocco. E per arrivarci si riparte da capo: non dai termini, ma da **che cosa si è autorizzati ad asserire**.

Quattro forme di giudizio

$\Gamma \text{ ctx}$	Γ è un contesto ben formato
$\Gamma \vdash A \text{ type}$	A è un tipo
$\Gamma \vdash a : A$	a è un elemento di A
$\Gamma \vdash a \equiv b : A$	a e b sono definizionalmente uguali

Il punto da non perdere

L'uguaglianza $\equiv$ è un **giudizio**: è decidibile, la controlla il kernel, non si dimostra. Presto incontreremo un'uguaglianza che è invece una **proposizione**, cioè un tipo abitabile o meno.

Come si costruisce un contesto

FORMAZIONE DEI CONTESTI

$$\frac{}{\diamond \text{ ctx}} \text{CTX-}\diamond \qquad \frac{\Gamma \text{ ctx} \quad \Gamma \vdash A \text{ type} \quad x \notin \text{dom}(\Gamma)}{\Gamma, x:A \text{ ctx}} \text{CTX-EXT}$$

ED È QUI CHE SERVE — LA REGOLA DELLA VARIABILE

$$\frac{\Gamma, x:A, \Delta \text{ ctx}}{\Gamma, x:A, \Delta \vdash x : A} \text{VAR}$$

Un contesto non è una lista di ipotesi

La premessa di CTX-EXT è $\Gamma \vdash A \text{ type}$ *nel contesto* Γ : ogni tipo può dipendere dalle variabili precedenti. Un contesto è un **telescopio**: $n:\mathbb{N}$, $v:\text{Vec } \mathbb{N} \ n$, $p: n > 0$.

Lo schema a quattro tempi

Ogni costruttore di tipi si presenta con quattro gruppi di regole:

1. **formazione**: quando il tipo è ben formato;
2. **introduzione**: come si costruiscono i suoi elementi;
3. **eliminazione**: come si usano i suoi elementi;
4. **computazione** (β): che cosa succede quando l'eliminazione incontra l'introduzione.

Nota

È la stessa disciplina della deduzione naturale (*harmony* fra intro ed elim). Riconoscerla rende meccanico leggere le regole di qualunque tipo, incluso quelli che definiremo noi in Lean.  L4: **inductive**, **cases**, **induction**

$$\frac{\Gamma \vdash A \text{ type} \quad \Gamma, x:A \vdash B \text{ type}}{\Gamma \vdash \Pi x:A. B \text{ type}} \Pi\text{-F}$$

$$\frac{\Gamma, x:A \vdash b : B}{\Gamma \vdash \lambda x. b : \Pi x:A. B} \Pi\text{-I}$$

$$\frac{\Gamma \vdash f : \Pi x:A. B \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B[x := a]} \Pi\text{-E}$$

$$\frac{\Gamma, x:A \vdash b : B \quad \Gamma \vdash a : A}{\Gamma \vdash (\lambda x. b) a \equiv b[x := a] : B[x := a]} \Pi\text{-}\beta$$

- ▶ se $x \notin \text{FV}(B)$ si riottiene $A \rightarrow B$;
- ▶ lettura logica: $\Pi x:A. B(x)$ è $\forall x:A. B(x)$.

Σ : coppie dipendenti = $\exists$

SINTASSI

$$A, B ::= \dots \mid \Sigma x:A. B \quad t ::= \dots \mid \langle a, b \rangle \mid \text{ind}_{\Sigma}(p, c)$$

$$\frac{\Gamma \vdash A \text{ type} \quad \Gamma, x:A \vdash B \text{ type}}{\Gamma \vdash \Sigma x:A. B \text{ type}} \Sigma\text{-F}$$

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B[x := a]}{\Gamma \vdash \langle a, b \rangle : \Sigma x:A. B} \Sigma\text{-I}$$

- ▶ in $\Sigma\text{-I}$ il secondo componente ha tipo $B[x := a]$: **dipende dal primo**. È tutta qui la differenza col prodotto;
- ▶ se $x \notin \text{FV}(B)$ si riottiene $A \times B$: la congiunzione di L1 è il caso non dipendente.

$$\frac{\Gamma \vdash p : \Sigma x:A. B \quad \Gamma, x:A, y:B \vdash c : C(\langle x, y \rangle)}{\Gamma \vdash \text{ind}_{\Sigma}(p, c) : C(p)} \quad \Sigma\text{-E}$$

$$\text{ind}_{\Sigma}(\langle a, b \rangle, c) \equiv c[x := a][y := b]$$

- ▶ per *usare* una coppia basta saper trattare il caso $\langle x, y \rangle$: è l'unico modo di costruirla
🔗 L1: esattamente come il *case* per $+$;
- ▶ C è il **motive**: una *famiglia* indicizzata da $p : \Sigma x:A. B$, non un tipo fisso. Con C costante si riottiene l'eliminazione ordinaria, cioè l'*uncurry*.

Nota

Il **motive** tornerà nel blocco 2, in ogni ricorsore generato dal kernel.

Le proiezioni, e un $\exists$ che consegna il testimone

Le proiezioni non sono primitive: si **derivano** dall'eliminatore.

$$\begin{array}{ll}\pi_1 p := \text{ind}_\Sigma(p, \lambda x. \lambda y. x) & : A \\ \pi_2 p := \text{ind}_\Sigma(p, \lambda x. \lambda y. y) & : B[x := \pi_1 p]\end{array}$$

Il tipo di π_2 menziona π_1 : per costruirlo serve l'eliminazione *dipendente*, con $C(p) := B[x := \pi_1 p]$.
Un eliminatore non dipendente darebbe solo π_1 .

$\Sigma x:A. B(x)$ è $\exists x:A. B(x)$, ma un $\exists$ **costruttivo**: π_1 estrae *sempre* il testimone.

💡 Assioma della scelta

In MLTT $\prod x:A. \Sigma y:B. R(x, y) \rightarrow \Sigma f:A \rightarrow B. \prod x:A. R(x, f x)$ è *dimostrabile*.
Perché non è «l'» assioma della scelta?

$$\frac{\Gamma \vdash A \text{ type} \quad \Gamma \vdash a : A \quad \Gamma \vdash b : A}{\Gamma \vdash \text{Id}_A(a, b) \text{ type}} \text{Id-F}$$

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{refl}_a : \text{Id}_A(a, a)} \text{Id-I}$$

ELIMINAZIONE — J, L'INDUZIONE SUI CAMMINI

$$\frac{\Gamma \vdash p : \text{Id}_A(a, b) \quad \Gamma, z:A \vdash c : C(z, z, \text{refl}_z)}{\Gamma \vdash J(c, p) : C(a, b, p)} \text{Id-E}$$

$$J(c, \text{refl}_a) \equiv c[z := a]$$

Il **motive** $C(x, y, p)$ dipende dai due estremi *e dal cammino*. E la premessa chiede c solo nel caso refl : **è l'unico costruttore, e basta saper trattare quello.**

Due uguaglianze, e non sono la stessa

	$a \equiv b : A$	$\text{Id}_A(a, b)$
che cos'è	giudizio	tipo (proposizione)
chi decide	il kernel, calcolando	si deve <i>dimostrare</i>
come si usa	silenziosamente, ovunque	con J
esempio	$2 + 2 \equiv 4$	$\forall n. 0 + n = n$

⚠ Dove le due nozioni divergono

Su termini *chiusi* coincidono. Con variabili libere no: $n + 0 \equiv n$ vale per definizione, ma $0 + n = n$ va dimostrato per induzione — dipende da quale argomento consuma la ricorsione.

🔗 L4: **rfl** contro **induction**: è esattamente questa differenza

$$\text{Type}_0 : \text{Type}_1 : \text{Type}_2 : \dots$$

- ▶ ogni tipo abita un universo; nessun universo abita se stesso;
- ▶ **cumulatività**: $A : \text{Type}_i$ implica $A : \text{Type}_j$ per $j \geq i$. Rocq ce l'ha, **Lean 4 no**;
- ▶ $\prod x:A. B$ con $A : \text{Type}_i, B : \text{Type}_j$ vive in $\text{Type}_{\max(i,j)}$ — **predicativo**.

In Lean il servizio della cumulatività lo rende il **polimorfismo di universo**: $\text{Type } u$ con u *variabile* di livello — una definizione sola, valida a ogni livello.

⚠ Perché serve

Con $\text{Type} : \text{Type}$ si riproduce Russell/Burali-Forti in veste type-theoretic: il **paradosso di Girard** [🔗](#) *L2: ci siamo appena stati*. La gerarchia è il prezzo della coerenza.

$$\frac{}{\vdash \text{zero} : \mathbb{N}} \qquad \frac{\vdash n : \mathbb{N}}{\vdash \text{succ } n : \mathbb{N}}$$

L'eliminatore (*ricorsore*) è il principio di induzione:

$$\text{ind}_{\mathbb{N}} : \Pi C : \mathbb{N} \rightarrow \mathcal{U}. C(\text{zero}) \rightarrow (\Pi n : \mathbb{N}. C(n) \rightarrow C(\text{succ } n)) \rightarrow \Pi n : \mathbb{N}. C(n)$$

$$\text{ind}_{\mathbb{N}}(C, c_0, c_s, \text{zero}) \equiv c_0 \qquad \text{ind}_{\mathbb{N}}(C, c_0, c_s, \text{succ } n) \equiv c_s \, n \, (\text{ind}_{\mathbb{N}}(C, c_0, c_s, n))$$

Nota

Scriverlo esplicitamente alla lavagna: in L4 questo diventa letteralmente **induction** `n with | zero => ... | succ n ih => ....` La `ih` è l'argomento $C(n)$ di c_s .

Bool

true, false : Bool

List A

nil, cons : $A \rightarrow \text{List } A \rightarrow \text{List } A$

Vec A n

(famiglia *indicizzata*)

vnil : Vec A 0

vcons : $A \rightarrow \text{Vec } A\ n \rightarrow \text{Vec } A\ (n+1)$

L'indice *dipende da un termine*: qui i tipi dipendenti guadagnano il loro stipendio.

Esercizio

Scrivere il tipo dell'eliminatore di List A. Confrontarlo con l'enunciato dell'induzione strutturale sulle liste.

- ▶ **normalizzazione** $\Rightarrow \equiv$ decidibile $\Rightarrow$ **type checking decidibile** — per la teoria *intensionale*; nell'estensionale no;
- ▶ **canonicità**: ogni termine chiuso di tipo $\mathbb{N}$ si riduce a un numerale — da qui *disgiunzione ed esistenza*;
- ▶ **Aczel** [Acz78]: la teoria degli insiemi costruttiva **CZF** si interpreta dentro MLTT, con un universo e i **W-tipi** — gli alberi ben fondati $\forall x:A. B(x)$;
- ▶ **Hofmann** [Hof95]: la teoria estensionale è *conservativa* su quella intensionale + K + **funext**.

CZF non è ZFC: intuizionista e *predicativa*, senza insieme potenza. Ma $\text{CZF} + \text{terzo escluso} = \text{ZF}$.

💡 Che cosa dicono, in due frasi

Aczel: MLTT non *assomiglia* a una teoria degli insiemi, ne *contiene* una.

Hofmann: l'estensionalità non compra potere, compra brevità.

Blocco 2

Il Calcolo delle Costruzioni Induttive

- ▶ **Calculus of Constructions**: prodotti dipendenti, Prop impredicativo, gerarchia Type_i predicativa  L2: il vertice λC ;
- ▶ + **scemi induttivi** alla Martin-Löf, con generazione automatica dei ricorsori (Paulin-Mohring);
- ▶ = il kernel di **Rocq** e, con alcune varianti, il type checker di **Lean 4**.

Varianti in Lean 4

Prop con *definitional proof irrelevance*; quozienti e **Classical.choice** come assiomi (Lean è coerente ma *non* puramente costruttivo di default); **Type** u con polimorfismo di universo esplicito.

Che cos'è uno schema induttivo

Un tipo induttivo si dichiara dando i **costruttori**; il kernel ne ricava tutto il resto.

```
inductive Vec ( $\alpha$  : Type u) : Nat  $\rightarrow$  Type u where    --  $\alpha$  parametro, Nat indice  
  | nil   : Vec  $\alpha$  0  
  | cons :  $\alpha \rightarrow$  Vec  $\alpha$  n  $\rightarrow$  Vec  $\alpha$  (n+1)
```

Parametri: fissi in tutti i costruttori. **Indici**: variano, e finiscono nel *motive* dell'eliminatore.

La u è una variabile di **livello**: `Vec` è dichiarato una volta e vive in ogni universo $\hookrightarrow$: *polimorfismo di universo*.

⚠ Stretta positività

Il tipo che si sta definendo può comparire negli argomenti dei costruttori solo in posizione **positiva**: mai a sinistra di una freccia. Senza questo vincolo il sistema crolla.

Perché serve la positività

```
inductive Bad where
  | mk : (Bad → False) → Bad      -- Bad a SINISTRA di →: rifiutato

def f : Bad → False
  | .mk g => g (.mk g)

example : False := f (.mk f)      -- ⊥, e la riduzione non termina
```

Lean rifiuta la dichiarazione: *non-positive occurrence of the datatypes being declared.*

💡 Intuizione

Un'occorrenza negativa permette l'auto-applicazione: è lo stesso meccanismo del λ -calcolo non tipato, dove $\Omega = (\lambda x. xx)(\lambda x. xx)$ non termina. La positività è ciò che garantisce che l'induttivo sia davvero il più piccolo punto fisso  L2: il minimo insieme chiuso.

Il ricorsore, generato dal kernel

```
Nat.rec : {motive : Nat → Sort u} →  
  motive Nat.zero →  
  ((n : Nat) → motive n → motive n.succ) →  
  (t : Nat) → motive t
```

- ▶ il **motive** è una famiglia $T \rightarrow \text{Sort } u$, non un tipo fisso: è questo a rendere l'eliminazione **dipendente**;
- ▶ motive costante $\Rightarrow$ si riottiene l'iteratore  L2: la codifica impredicativa si fermava qui;
- ▶ motive in Prop: è il *principio di induzione*; in Type: è la ricorsione che costruisce dati.

Regola di computazione (ι): $\text{Nat.rec } z \ s \ (\text{succ } n) \equiv s \ n \ (\text{Nat.rec } z \ s \ n)$. In Lean **induction, cases** e **match** sono zucchero sopra **Nat.rec**.  L4: è letteralmente la stessa cosa

Anche l'uguaglianza è un tipo induttivo

```
inductive Eq {α : Sort u} (a : α) : α → Prop where -- a e' parametro, il
    secondo indice
    | refl : Eq a a
```

Un solo costruttore. Il ricorsore che ne esce è la regola J del blocco 1:

```
Eq.rec : {motive : (b : α) → a = b → Sort v} →
    motive a rfl → {b : α} → (h : a = b) → motive b h
```

- **rfl** è l'unico modo di *costruire* un'uguaglianza; **rw**, **subst** e l'operatore ► sono tutti usi di **Eq.rec**;
- il tipo identità non è un'aggiunta esotica: è un induttivo come $\mathbb{N}$ o **List**.

E poiché il costruttore è uno solo, nulla obbliga $\text{Id}_A(a, b)$ ad avere *al più* un abitante: è la porta da cui entra HoTT, fra due slide.

Prop contro Type: che cosa si può eliminare

In CIC un induttivo in Prop **non** può essere eliminato verso Type: da una prova non si estraggono dati.

Tipo	Elim. verso Type	Perché
False, Eq, And	permessa	al più un costruttore, nessun dato
Or, Exists	vietata	sceglierne uno rivelerebbe informazione

i Le due facce della restrizione

Senza di essa, impredicatività + eliminazione forte = incoerenza [L2: Coquand 1986](#).
Con essa si guadagna l'**estrazione**: le prove si cancellano, e resta il programma. È anche il motivo del **noncomputable** di Lean. [L4: *Classical.choice*](#)

MLTT vs CoC vs CIC

	MLTT	CoC	CIC
prodotti dipendenti II	✓	✓	✓
universo impredicativo	✗	✓ (Prop)	✓ (Prop)
gerarchia predicativa	✓	✓	✓
tipi induttivi primitivi	✓	✗ (codifiche)	✓
proof irrelevance	✗	(opzionale)	✓ (Lean)
implementazioni	Agda, Idris	—	Rocq, Lean 4

In CoC i tipi induttivi si *codificano* (impredicativamente, alla Church) ma senza le regole η /dipendenti che servono per l'induzione: di qui la necessità di primitivizzarli.

I kernel veri: Rocq e Lean 4

	Rocq	Lean 4
Prop	impredicativo; proof irrelevance <i>proposizionale</i> (o SProp)	impredicativo; proof irrelevance definizionale
universi	cumulativi; polimorfismo opt-in	non cumulativi; Sort u ovunque
quozienti	setoidi, o assiomi	primitivi ($\Rightarrow$ funext è teorema)
assiomi	nessuno di serie	propext , Classical.choice , Quot.sound

💡 Il de Bruijn criterion, in pratica

Entrambi hanno un kernel piccolo e indipendente dal resto 🌀 *L1: tattiche e IA non sono nella base fidata*, e per entrambi esistono *type-checker esterni* scritti da terzi: la fiducia si può replicare.

La metateoria: perché il kernel può esistere

$SN \implies \equiv \text{decidibile} \implies \text{type checking decidibile} \implies \text{un kernel}$

- ▶ vale per la teoria **intensionale**: in quella estensionale la riflessione dell'uguaglianza rende il controllo indecidibile;
- ▶ la SN di CIC non è dimostrabile *dentro* CIC  L2: *ancora Gödel*; la dimostrazione è nella tesi di Werner, 1994.

CANONICITÀ

Ogni termine chiuso di tipo $\mathbb{N}$ si riduce a un numerale. Di qui la **proprietà della disgiunzione** e quella **dell'esistenza**: da $\vdash \exists x. P(x)$ si estrae un testimone.

Quanto è forte

Quanto a coerenza, CIC con la gerarchia di universi sta a ZFC con una successione di inaccessibili.  : *fra due slide: che aspetto ha quel modello, e perché esiste*

Il polimorfismo non è insiemistico

Che cosa è un tipo? La risposta ingenua: un insieme, e $A \rightarrow B$ l'insieme di **tutte** le funzioni da A a B . Ma questo porta a un assurdo: Reynolds [Rey84] dimostra che non esiste un modello ingenuo di System F, e quindi nemmeno di λC .

SCHEMA DELLA DIMOSTRAZIONE — Reynolds [Rey84]

- ▶ in L2: $\forall X. (FX \rightarrow X) \rightarrow X$ è l'**algebra iniziale** di F , per ogni F definibile;
- ▶ prendi $F(X) = (X \rightarrow 2) \rightarrow 2$, cioè $\wp\wp$;
- ▶ un'algebra iniziale è un punto fisso (Lambek): $A \cong \wp(\wp A)$;
- ▶ ma $|A| < |\wp A| < |\wp\wp A|$ per Cantor. **Assurdo**.

λC lo eredita perché contiene System F dentro Prop.

💡 Che cosa si fa allora

Si tengono gli insiemi e si restringe la *freccia*: PER su $\mathbb{N}$, insiemi modesti, domini. In $A \rightarrow B$ stanno solo le funzioni *realizzabili*, e $\forall$ è un'intersezione: resta piccola.

E allora perché Lean un modello ce l'ha?

Eppure Carneiro [Car19] ne costruisce uno, in **ZFC + ω inaccessibili**. Come?

PRIMA MOSSA — L'IMPREDICATIVO È Prop, E Prop È PROOF-IRRELEVANT

Una proposizione denota **al più un elemento**: $\emptyset$ se falsa, $\{\emptyset\}$ se vera. Quindi $\forall X:\text{Prop}. X$ è un'intersezione di sottosingoletti: **nessuna cardinalità esplode**.

SECONDA MOSSA — Type È PREDICATIVO

$\text{Type}_U := V_{\kappa_U}$ con κ_U inaccessibile: i Π non escono dall'universo, perché un inaccessibile è chiuso per prodotti di taglia minore.

💡 Intuizione

La «seconda guardia» di L2 — niente eliminazione da Prop verso Type — è *esattamente* ciò che rende lecito il collasso: se non puoi osservare *quale* prova avevi, il modello le identifica tutte.

Che cosa MLTT *non* dimostra

L'ASSIOMA K — unicità delle prove di identità

$$K : \prod a:A. \prod p:\text{Id}_A(a, a). \quad p = \text{refl}_a$$

- ▶ **non** è derivabile: Hofmann e Streicher (1998) costruiscono il **modello dei gruppoidi**, dove un tipo è un gruppoide e Id l'insieme degli isomorfismi;
- ▶ lì $\text{Id}_A(a, a)$ può avere *più* abitanti: K è falso nel modello, dunque non è un teorema.

E L'ESTENSIONALITÀ FUNZIONALE

Nemmeno **funext** è derivabile. In Rocq è un assioma; in Lean è un *teorema*, perché i quozienti sono primitivi e lo si ricava da **Quot.sound**.

Non sono lacune: Id ha una struttura più ricca di quanto sembri

Un seme per il futuro: Homotopic Type Theory (HoTT)

- ▶ che cosa è $\text{Id}_A(a, b)$? In MLTT non è necessariamente un tipo con al più un abitante;
- ▶ interpretazione omotopica: A è uno *spazio*, a, b punti, $\text{Id}_A(a, b)$ lo spazio dei *cammini* da a a b , e $\text{Id}_{\text{Id}_A(a, b)}(p, q)$ le omotopie fra cammini...
- ▶ **univalenza** (Voevodsky): $(A \simeq B) \simeq \text{Id}_{\mathcal{U}}(A, B)$ — «tipi equivalenti sono uguali».

💡 Intuizione

Non lo useremo. Ma è lo sviluppo naturale di ciò che abbiamo appena visto, ed è il motivo per cui vale la pena distinguere con cura $\equiv$ da Id .

- ▶ P. Martin-Löf. *Intuitionistic Type Theory*. Studies in Proof Theory. Note di Giovanni Sambin delle lezioni di Padova, 1980. Napoli: Bibliopolis, 1984
- ▶ B. Nordström, K. Petersson e J. M. Smith. *Programming in Martin-Löf's Type Theory: An Introduction*. International Series of Monographs on Computer Science. Oxford University Press, 1990
- ▶ C. Paulin-Mohring. «Inductive Definitions in the System Coq: Rules and Properties». In: *Typed Lambda Calculi and Applications (TLCA '93)*. Vol. 664. LNCS. Springer, 1993, pp. 328–345
- ▶ The Lean Developers. *The Lean Language Reference*. 2025. URL: <https://lean-lang.org/doc/reference/latest/> — sezione sul kernel.

- ▶ P. Aczel. «**The Type Theoretic Interpretation of Constructive Set Theory**». In: *Logic Colloquium '77*. A cura di A. Macintyre, L. Pacholski e J. Paris. North-Holland, 1978, pp. 55–66
- ▶ M. Hofmann. «**Extensional Concepts in Intensional Type Theory**». Pubblicata da Springer, Distinguished Dissertations, 1997. Tesi di dott. University of Edinburgh, 1995
- ▶ B. Werner. «**Une Théorie des Constructions Inductives**». Tesi di dott. Université Paris VII, 1994 — la metateoria di CIC.
- ▶ J. C. Reynolds. «**Polymorphism is not Set-Theoretic**». In: *Semantics of Data Types*. Vol. 173. LNCS. Springer, 1984, pp. 145–156

- ▶ B. Werner. «Sets in Types, Types in Sets». In: *Theoretical Aspects of Computer Software (TACS '97)*. Vol. 1281. LNCS. Springer, 1997, pp. 530–546 — il modello insiemistico di CIC.
- ▶ M. Carneiro. «The Type Theory of Lean». Tesi di laurea mag. Carnegie Mellon University, 2019 — e quello di Lean.
- ▶ M. Hofmann e T. Streicher. «The Groupoid Interpretation of Type Theory». In: *Twenty-Five Years of Constructive Type Theory*. Vol. 36. Oxford Logic Guides. Oxford University Press, 1998, pp. 83–111
- ▶ The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Princeton: Institute for Advanced Study, 2013. URL: <https://homotopytypetheory.org/book/> — per chi vuole seguire il seme.